



Figure 3: Thin/thick JavaFX client

JavaFX, with its strong integration with Java, WebView and animation features, is a very good option for cross-platform desktop application development. For example, a space research company like NASA that needs to analyse a large amount of data to display celestial objects and their movements, cannot be supported on the Web. HTML5 and JavaFX address different segments of applications with a little overlap.

Developing applications with JavaFX

JavaFX can be used in all types of business applications including those with databases at the server side as the backend, as well as Internet applications with thin and thick clients. In all the cases, server side implementations can be in Java, with the JavaFX client communicating with the server using network protocols. It can also be used to create a media centre. JavaFX applications run on desktops on top of the native operating system with a compatible JRE. To overcome the necessity of JRE, JavaFX also provides a tool that packages the necessary Java Runtime to the application, which can be deployed independently.

JavaFX makes more sense when there is a need for highly customised interfaces and large data management requirements, especially for enterprise applications intended for focused user sets. The high requirement of data visualisation involves way too much information for a browser to manage. The beauty of JavaFX is its ability to build these software tools with a small, modern, manageable code base. There have been a large number of successful commercial applications developed and deployed in the field of healthcare, automotive, hospitals and finance.

An example of JavaFX code using WebView

The following example illustrates the WebView feature of JavaFX, which opens a mini browser and displays the oracle.com site in it. The source code below should be saved in the file `WebViewSample.java`, compiled using

`javac (javac WebViewSample.java)` and run using Java (`java WebViewSample`).

```
import javafx.application.Application;
import javafx.geometry.HPos;
import javafx.geometry.VPos;
import javafx.scene.Node;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import javafx.scene.paint.Color;
import javafx.scene.web.WebEngine;
import javafx.scene.web.WebView;
import javafx.stage.Stage;

public class WebViewSample extends Application {
    private Scene scene;
    @Override public void start(Stage stage) {
        // create the scene
        stage.setTitle("Web View");
        scene = new Scene(new Browser(), 900, 600);
        stage.setScene(scene);
        scene.getStylesheets().add("webviewsample/
BrowserToolbar.css");
        stage.show();
    }
    public static void main(String[] args){
        launch(args);
    }
}

class Browser extends Region {
    final WebView browser = new WebView();
    final WebEngine webEngine = browser.getEngine();

    public Browser() {
        //apply the styles
        getStyleClass().add("browser");
        // load the web page
        webEngine.load("http://www.oracle.com/products/index.
html");
        //add the web view to the scene
        getChildren().add(browser);
    }
    private Node createSpacer() {
        Region spacer = new Region();
        HBox.setHgrow(spacer, Priority.ALWAYS);
        return spacer;
    }
    @Override protected void layoutChildren() {
        double w = getWidth();
        double h = getHeight();
        layoutInArea(browser, 0, 0, w, h, HPos.CENTER, VPos.
CENTER);
    }
}
```